Practical No. 10

Name:- Darshan Chavan

Roll no:- A-16

Branch :- SY-AI&DS

Subject :- PDS

Aim - Version Control using Gits

1. What is the difference between Git and GitHub?

Git:

- Git is a Version Control System (VCS)
- Created by Linus Torvalds in 2005
- Works completely offline on your local machine
- Tracks every change made to your code
- Stores full project history locally
- Free and open-source

GitHub:

- GitHub is a cloud-based hosting platform for Git repos
- Founded in 2008, bought by Microsoft in 2018
- Requires internet connection
- Adds collaboration features: Pull Requests, Issues, Actions
- Stores your code remotely on servers
- Has free and paid plans
- Alternatives: GitLab, Bitbucket

2. Difference Between git add git commit

Git Add:
The git add command is the first step in saving changes in Git. When you make edits to files in your working directory, those changes are not automatically tracked for saving. By using git add, you select which files or modifications should be prepared for the next commit. This process moves the changes into the *staging area*, which acts like a holding zone. It does not permanently save anything yet, but it tells Git, “These are the files I want to include in my next snapshot.” You can add individual files, multiple files, or even all changes at once using commands like git add file.txt or git add .. In short, git add is about choosing and preparing changes, not recording them.

Git Commit:
The git commit command is the step that actually records your work into the project’s history. Once changes are staged with git add, committing takes those changes and creates a permanent snapshot in the repository. Each commit is saved with a unique ID and usually includes a message that describes what was changed, such as git commit -m "Fixed login bug". This message helps you and others understand the purpose of the commit later. Unlike git add, which only prepares changes, git commit finalizes them and makes them part of the timeline of the project. Over time, commits build up into a detailed history of how the project has evolved, making it possible to track progress, undo mistakes, or collaborate with others effectively.

3. What is the staging area in Git?

The staging area in Git, also called the *index*, is a special space where changes are collected and prepared before they are permanently recorded in the repository. When you edit files in your working directory, those changes are not automatically saved into Git’s history. Instead, you use the `git add` command to move selected changes into the staging area. This acts like a middle ground between your working directory and the repository. The staging area allows you to carefully choose which changes should be included in the next commit, and which ones should be left out for later. For example, if you modify three files but only want to commit two of them, you can add just those two files to the staging area. Once the staging area contains the desired changes, you use `git commit` to take a snapshot of everything staged and save it permanently in the repository’s history. In this way, the staging area gives you control and flexibility, letting you organize and group changes logically before committing them. It is one of Git’s most powerful features because it allows developers to build clean, meaningful commits rather than saving every single change immediately.

4. What happens if you modify a file after committing it?

When you commit a file in Git, you are saving a snapshot of its current state into the repository's history. If you later modify that same file, the changes you make are not automatically included in the commit you already created. Instead, the file becomes "modified" again in your working directory. Git will detect that the file has changed compared to the last committed version, but those new changes are not yet staged or saved. To include the new modifications in the repository's history, you must go through the process again: first use git add to move the updated file into the staging area, and then use git commit to record a new snapshot. This means that every time you edit a file after committing, you are essentially creating a new version that can be tracked separately. The earlier commit remains unchanged in history, while the new commit adds another layer of progress. This system allows developers to keep a clear timeline of how files evolve over time, making it possible to roll back to earlier versions if needed, compare differences between commits, or understand the sequence of changes. In short, modifying a file after committing does not overwrite the old commit; it simply creates new changes that must be staged and committed again to become part of the project's permanent history.

5. Difference between git push git pull?

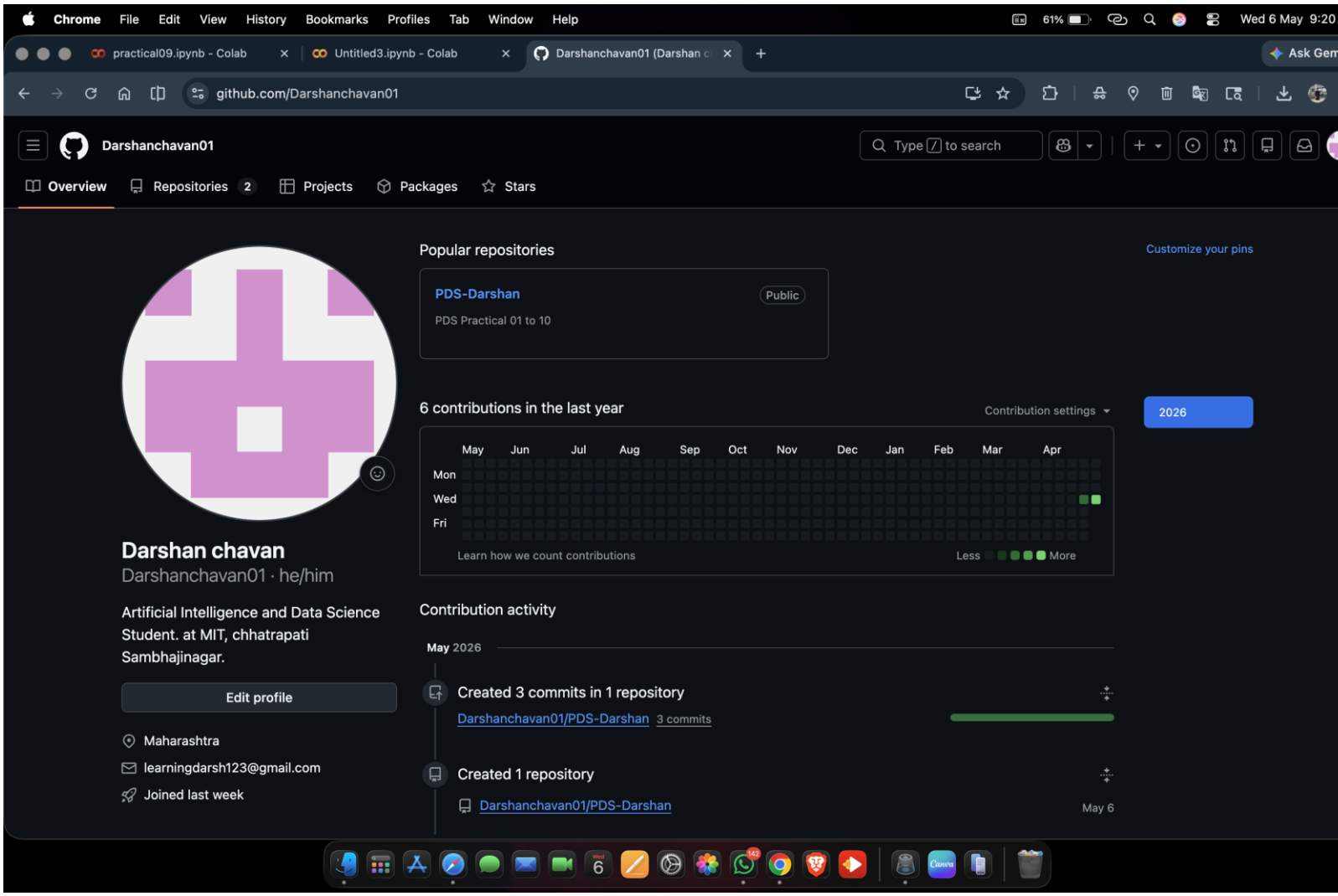
Git Push:
The git push command is used to send your local commits to a remote repository such as GitHub, GitLab, or Bitbucket. When you make changes in your project, you first add them to the staging area with git add and then save them into your local repository with git commit. At this point, the changes exist only on your computer. To share your work with others or to keep the remote repository updated, you use git push. This command uploads all the commits from your local branch to the corresponding branch on the remote server. It ensures that your contributions are visible to teammates and integrated into the shared project. In collaborative environments, pushing is essential because it allows everyone to access the latest version of the codebase. Without git push, your work remains private and disconnected from the team's repository. In short, git push is the action of sending your local history outward to the remote repository so that it becomes part of the collective project

Git Pull:
The git pull command performs the opposite role: it brings changes from the remote repository into your local repository. When other developers push their commits, your local copy may become outdated. Running git pull downloads the latest commits from the remote branch and merges them into your local branch. This keeps your project synchronized with the team's work. Essentially, git pull is a combination of two commands: git fetch, which retrieves the new data from the remote, and git merge, which integrates that data into your current branch. By pulling regularly, you avoid conflicts and ensure that your local environment reflects the most recent state of the project. In collaborative workflows, pulling is just as important as pushing because it keeps everyone aligned and prevents situations where multiple people are working on outdated versions of the code.

6 .What is Version Control? Differentiate between Centralized v/s Distributed VCS?

Version Control:
Version Control is a system that helps developers manage changes to source code or documents over time. It records every modification made to files, allowing you to track history, revert to earlier versions, and collaborate with others without losing work. With version control, multiple people can work on the same project simultaneously, and the system keeps track of who made which changes and when. This makes teamwork more organized and reduces the risk of overwriting or losing important updates. In software development, version control is essential because it ensures that projects evolve in a structured way, with a complete record of progress and the ability to roll back if mistakes occur. Git is one of the most popular version control systems used today.

Centralized vs Distributed Version Control Systems:
In a Centralized Version Control System (CVCS), there is a single central server that stores all the versions of the project. Developers connect to this server to commit changes or to update their local copy. Examples include Subversion (SVN) and CVS. The advantage of CVCS is simplicity and a clear single source of truth, but the drawback is that if the central server goes down, developers cannot commit or collaborate until it is restored. On the other hand, a Distributed Version Control System (DVCS), such as Git or Mercurial, does not rely on a single central server. Instead, every developer has a full copy of the repository, including its entire history, on their local machine. This means they can commit, branch, and view history even without internet access. Collaboration happens by pushing and pulling changes between repositories. DVCS offers more flexibility, faster operations, and resilience because work can continue even if the main server is unavailable. In short, CVCS depends on one central repository, while DVCS gives every developer their own complete copy, making it more powerful and fault-tolerant.


Chrome File Edit View History Bookmarks Profiles Tab Window Help
Wed 6 May 9:20
practical09.ipynb - Colab
Untitled3.ipynb - Colab
Darshanchavan01 (Darshan
Ask Gem
github.com/Darshanchavan01
Darshanchavan01
Type / to search
Overview
Repositories 2
Projects
Packages
Stars
Popular repositories
Customize your pins
PDS-Darshan
Public
PDS Practical 01 to 10
6 contributions in the last year
Contribution settings
2026
May Jun Jul Aug Sep Oct Nov Dec Jan Feb Mar Apr
Mon
Wed
Fri
Learn how we count contributions
Less More
Darshan chavan
Darshanchavan01 · he/him
Artificial Intelligence and Data Science Student. at MIT, chhatrapati Sambhajinagar.
Edit profile
Maharashtra
learningdarsh123@gmail.com
Joined last week
Contribution activity
May 2026
Created 3 commits in 1 repository
Darshanchavan01/PDS-Darshan 3 commits
Created 1 repository
Darshanchavan01/PDS-Darshan
May 6

Chrome File Edit View History Bookmarks Profiles Tab Window Help 61% Wed 6 May 9:20 AM

practical09.ipynb - Colab | Untitled3.ipynb - Colab | Darshanchavan01/PDS-Darsh | Ask Gemini

github.com/Darshanchavan01/PDS-Darshan

Darshanchavan01 / PDS-Darshan

Type / to search

Code | Issues | Pull requests | Actions | Projects | Wiki | Security and quality | Insights | Settings

PDS-Darshan Public

Pin | Watch 0 | Fork 0 | Star 0

main | 1 Branch | 0 Tags | Go to file T | Add file | Code

Darshanchavan01 PDS 8f96bec · 4 minutes ago 3 Commits

Darshanpractical09.ipynb - Colab.pdf	Add files via upload	7 minutes ago
PDSpractical02 (1).pdf	PDS	4 minutes ago
Practicaletheory01.ipynb - Colab.pdf	PDS	4 minutes ago
README.md	Initial commit	8 minutes ago
darshanprac4 (1).pdf	PDS	4 minutes ago
newpracticaltheory03.ipynb - Colab.pdf	PDS	4 minutes ago
practical06.ipynb - Colab.pdf	PDS	4 minutes ago
practical08.ipynb - Colab.pdf	PDS	4 minutes ago
practicle no 5 Darshan (1).pdf	PDS	4 minutes ago

README

PDS-Darshan

About

PDS Practical 01 to 10

Readme
Activity
0 stars
0 watching
0 forks

Releases

No releases published
Create a new release

Packages

No packages published
Publish your first package

Contributors 1

Darshanchavan01 Darshan chavan

₹